



Java Class Attributes

[< Previous](#)[Next >](#)

Java Class Attributes

In the previous chapter, we used the term "**variable**" for **x** in the example (as shown below).

In Java, variables declared inside a class are called "**attributes**".

You can also say that attributes are variables that belong to a class:

Create a class called "**Main**" with two attributes: **x** and **y**:

```
public class Main {  
    int x = 5;  
    int y = 3;  
}
```

Another name for attributes is **fields**.

Accessing Attributes

You can access attributes by creating an object of the class, and by using the dot syntax (**.**):

The following example will create an object of the **Main** class, with the name **myObj**. We use the **x** attribute on the object to print its value:



```
public class Main {  
    int x = 5;  
  
    public static void main(String[] args) {  
  
  
    }  
}
```

Try it Yourself »

Modify Attributes

You can also modify attribute values:

Example

Set the value of `x` to 40:

```
public class Main {  
    int x;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
  
        System.out.println(myObj.x);  
    }  
}
```

Try it Yourself »



Example

Change the value of **x** to 25:

```
public class Main {  
    int x = 10;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
  
        System.out.println(myObj.x);  
    }  
}
```

Try it Yourself »

If you don't want the ability to override existing values, declare the attribute as **final**:

Example

```
public class Main {  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        myObj.x = 25; // will generate an error: cannot assign a value to a  
        System.out.println(myObj.x);  
    }  
}
```

Try it Yourself »

[REMOVE ADS](#)

Multiple Objects

If you create multiple objects of one class, you can change the attribute values in one object, without affecting the attribute values in the other:

Example

Change the value of `x` to 25 in `myObj2`, and leave `x` in `myObj1` unchanged:

```
public class Main {  
    int x = 5;  
  
    public static void main(String[] args) {  
        Main myObj1 = new Main(); // Object 1  
        Main myObj2 = new Main(); // Object 2  
        myObj2.x = 25;  
        System.out.println(myObj1.x); // Outputs 5  
        System.out.println(myObj2.x); // Outputs 25  
    }  
}
```

Try it Yourself »

Multiple Attributes

You can specify as many attributes as you want:



```
public class Main {
    String fname = "John";
    String lname = "Doe";
    int age = 24;

    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println("Name: " + myObj.fname + " " + myObj.lname);
        System.out.println("Age: " + myObj.age);
    }
}
```

Try it Yourself »

The next chapter will teach you how to create class methods and how to access them with objects.

Exercise [?]

Create an object and print its attribute value (500).

```
class Main {
    int x = 500;
}

public class Program {
    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println(myObj.x);
    }
}
```